

CODIFICACIÓN DE FUENTE

Ejercicio PPM:

- 1- Los contextos que se crean son los siguientes:

Orden 2:

Contexto (AB) y (BR)

Orden 1:

Contexto (A), (B) y (R)

- 2- 1er Evento:

Leemos A: ESC ESC ESC A(1/64)

2do Evento:

Leemos B: ESC ESC ESC B(1/64)

3er Evento:

Leemos R: ESC ESC ESC R(1/64)

4to Evento:

Leemos A: ESC ESC A(1/6)

- 3- En dicho caso que tengamos un alfabeto de 32 símbolos, y las probabilidades de cada símbolo sean equiprobables. Entonces, el tamaño del intervalo final sería la cantidad de bits en las cuales pueden entrar los 32 símbolos.

De esta forma, utilizando la siguiente fórmula:

$$\text{Número de bits} = \lceil \log_2 (\text{Cantidad de símbolos}) \rceil$$

Obtendríamos los bits necesarios para el intervalo, obteniendo el tamaño del intervalo.

$$\text{Número de bits} = \lceil \log_2 (32) \rceil$$

De manera que, con 5 bits, podemos representar el intervalo.

- 4- Para garantizar que las codificaciones de Huffman, Fano y Shannon coincidan, se realiza a través de distribuciones equiprobables, de manera, que, si cada evento tiene la misma probabilidad, los tres métodos van a generar las mismas longitudes de código.

Es así que dada la siguiente distribución:

LU= 1/7

MA= 1/7

MI= 1/7

JU= 1/7

VI= 1/7

SA= 1/7

DO= 1/7

Obtendríamos mismas longitudes de codificaciones.

Otra distribución que nos puede dar que las longitudes en los tres métodos de codificación sean similares, es en base a potencias de 2. Ya que de esta manera, por las características de los métodos, los de mayor valor, tendrían códigos de longitud más pequeños y los de menor valor, de mayor longitud.

Es así que se ofrece la siguiente distribución:

LU= 1/2

MA= 1/4

MI= 1/16

JU= 1/16

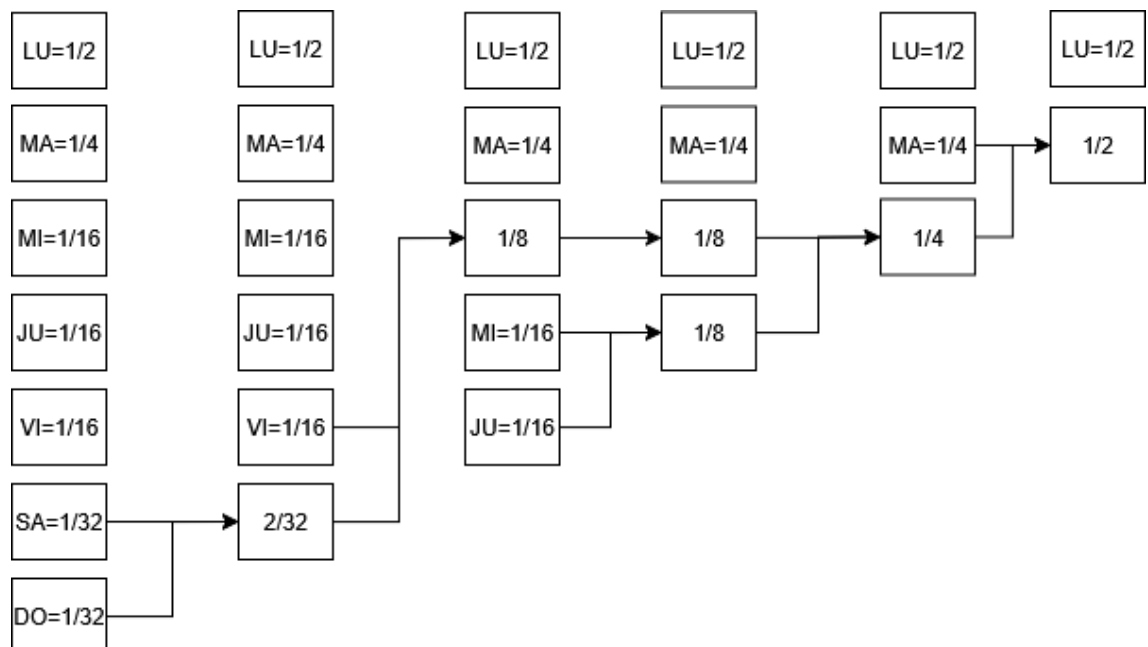
VI= 1/16

SA= 1/32

DO= 1/32

Comprobando la segunda distribución, primero obtenemos Huffman.

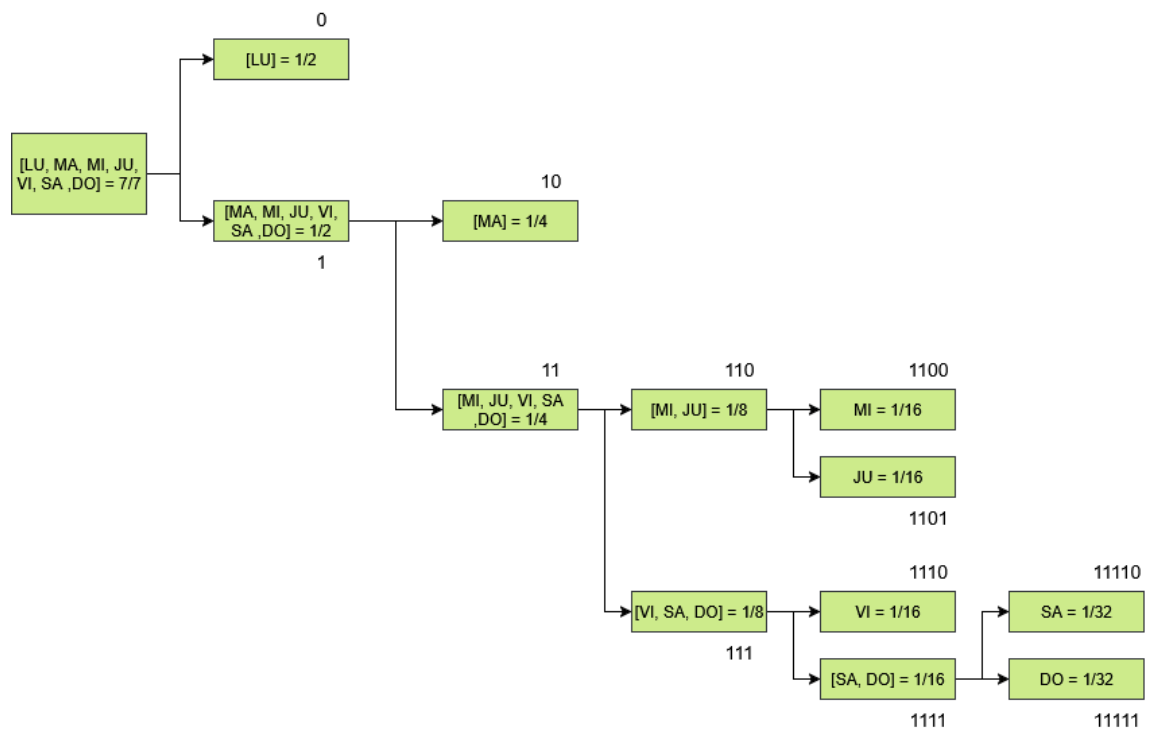
Para Huffman, aplicamos la codificación y obtenemos el árbol:



Símbolo	Código
LU	1
MA	00
MI	0110
JU	0111

VI	0100
SA	01010
DO	01011

Luego, Fano, también al ser equiprobables, al separar en r-grupos de similar probabilidad, las distancias terminan siendo de la misma longitud, demostrándolo por el siguiente gráfico:



Y por último Shannon, aplicando el teorema, el cuál dice que:

$$\log_2\left(\frac{1}{p_i}\right) \leq l_i \leq \log_2\left(\frac{1}{p_i}\right) + 1$$

Símbolo	p(s _i)	FA (previos)	Decimal	Binario	Longitud	Código
LU	½	0	0.000	0.000	1	0
MA	¼	½	0.5	0.100	2	10
MI	1/16	¾	0.750	0.110	4	110
JU	1/16	13/16	0.8125	0.1101	4	1101

VI	1/16	14/16	0.875	0.1110	4	1110
SA	1/32	15/16	0.9375	0.11110	5	11110
DO	1/32	31/32	0.96785	0.11111	5	11111

Como vemos, para todas las codificaciones obtuvimos las mismas longitudes y por lo tanto las mismas longitudes promedio. Pueden variar en cuánto a códigos, pero las longitudes son las mismas.

Enfocándose en los símbolos de mayor frecuencia que tengan una menor longitud y los de menor frecuencia, un código de mayor longitud.

CANAL DE COMUNICACIÓN

- a) Un canal es simétrico, si las probabilidades de error son iguales para ambos símbolos transmitidos, es decir, la probabilidad de error de 0, y la probabilidad de error de 1, deben ser iguales.

Verificando en el canal dado:

Canal	0	1
0	0.95	0.3
1	0.05	0.7

Como vemos, la probabilidad de error para 0 es:

$$p(1|0) = 1 - p(0|0) = 1 - 0.95 = 0.05$$

Y para 1 es:

$$p(0|1) = 1 - p(1|1) = 1 - 0.7 = 0.3$$

Dado que no son iguales, entonces no es simétrico.

- b) Para garantizar transmitir a la capacidad del canal, o maximizar la capacidad del canal, se debería transmitir el símbolo con mayor probabilidad.

Dado que es el que tiene menor error, y por lo tanto se aprovecharía más el canal y se reduciría la cantidad de errores.

Como vemos, probabilidad del símbolo 0, es mayor, por lo que convendría transmitir el símbolo 0 con mayor probabilidad.

- c) Software,

Para calcular la capacidad de un canal binario, utilizamos la fórmula:

$$C = 1 - H(p)$$

Donde C es la capacidad del canal binario, $H(p)$ es la entropía binaria, y p es la probabilidad de que un bit transmitido cambie, es decir, es la probabilidad de error.

Así, primero obteniendo:

$$\text{Probabilidad de error} = p(e) = \frac{0.05 + 0.3}{2} = 0.175$$

Ahora, aplicando la fórmula de la entropía binaria:

$$H(p) = p(e) * \log_2\left(\frac{1}{p(e)}\right) + (1 - p(e)) * \log_2\left(\frac{1}{1 - p(e)}\right)$$

$$H(p) = 0.175 * \log_2\left(\frac{1}{0.175}\right) + (1 - 0.175) * \log_2\left(\frac{1}{1 - 0.175}\right)$$

$$H(p) = 0.175 * 2.515 + 0.825 * 0.277 = 0.66$$

Entonces ahora aplicando en C

$$C = 1 - 0.66865 = 0.33135 \text{ bits/símbolo}$$

Capacidad del canal 0.33135 bits/símbolo.

DISCRETIZACIÓN

- a) Dado el rango de la señal origen que es -0.5v y +0.5v, nuestra señal oscila entre los 10v, lo cual, su rango dinámico es de 10v.
Dado el número de bits, el cual es 10, podemos obtener la cantidad de niveles en los que se divide el rango, tal es, $2^{10}=1024$
Así, obtenemos la resolución:

$$\text{Resolución} = \frac{\text{Rango}}{\text{Número de niveles}} = \frac{10}{1024} = 0.00977v$$

El error de discretización, es la diferencia entre la señal real y el valor discretizado. El error, se encuentra entre 0 y la resolución obtenida. Ya que, la señal puede ser positiva o negativa, el error máximo de discretización es la resolución dividida en 2.

$$\text{Error máximo de discretización} = \frac{\text{Resolución}}{2} = \frac{0.00977v}{2} = 0.00488v$$

- b) Dado el rango dinámico de 1v codificando con ADPCM, y queremos que el error máximo de discretización sea igual o menor al PCM original, es decir, 0.00488v, entonces si dividimos el rango en el error máximo deseado, obtendremos el aproximado de niveles necesarios para mantener el error del PCM original.

Así, calculamos:

$$\begin{aligned} \text{Número de niveles} &= \frac{\text{Rango dinámico}}{\text{Error máximo de cuantificación}} = \frac{1}{0.00488v} \\ &= 205.2 \end{aligned}$$

Redondeando, serán necesarios al menos, 206 niveles para mantener el error.

Ahora, lo que queremos saber son los bits, así que simplemente aplicamos $\log_2$ al número de niveles y obtenemos los bits necesarios para mantener el error.

$$\log_2(206) = 7.687 \cong 8 \text{ bits}$$

Necesitaríamos, aproximadamente 8 bits.